

gaugefit — 연동 가이드

gaugefit을 자기 애플리케이션 안에 넣는 방법. 계측학을 읽으려는 사람이 아니라, 라인에서 동작하게 만들어야 하는 개발자를 위해 쓴 문서다. 0.1.0 기준.

영문판은 [docs/integration.md](#)에 있고 내용은 같다.

1. gaugefit이 해 주는 일

영상과, 에지를 가로지르는 사각형 하나를 주면 된다. 그 에지가 어디인지를 수천분의 1 픽셀 단위로 알려 주고, 언제 그 답을 믿으면 안 되는지도 알려 준다.

측정 하나 위에 실제 검사가 필요로 하는 것들이 얹혀 있다.

- **페어링 정책** — 어느 두 에지가 폭을 이루는지를 추측이 아니라 명시로. 현장에서 나오는 오측정의 대부분이 이 선택이 감춰져 있는 데서 온다.
- **Line / Circle Finder** — 캘리퍼 여러 개, 개별 실패는 아웃라이어로 흡수하되 기록한다. 조용히 버리지 않는다.
- **미터올로지** — 피팅된 형상 사이의 거리, 각도, 평행도, 진원도, 직진도.
- **픽스처링** — 설정 전체가 프레임이 아니라 부품을 따라가게 하는 변환.
- **캘리브레이션** — 픽셀에서 밀리미터로. 영상을 리샘플링하는 게 아니라 측정된 점에 적용.
- **오차예산** — 숫자에 딸려 가는 불확실도. 어림짐작이 아니라 실측한 바이어스 곡선에 근거.

런타임 의존성은 numpy 하나다.

2. 설치

```
pip install gaugefit
```

Windows x86-64, Linux x86-64 / aarch64, macOS arm64에서 파이썬 3.10 ~ 3.14.

```
import gaugefit as gauge
gauge.engine_info()
# {'engine': 'native', 'native_available': True, 'simd': 'AVX2+FMA', 'threads': 32, ...}
```

영상은 2차원 실수 배열이다. numpy 모양이면 뭐든 되고, `gauge.imread()`가 PNG와 PGM을 의존성 없이 읽는다. 다른 포맷은 Pillow가 있으면 그쪽으로 간다. 값은 0~1이든 0~255든 상관없다 — 캘리퍼는 차이만 보기 때문이다. 다만 `contrast` 문턱값은 넣어 준 단위 그대로이니 하나를 정하고 지킬 것.

3. 5분짜리 버전

```
import gaugefit as gauge
img = gauge.imread("part.png")
cal = gauge.Caliper(center=(320, 240), angle_deg=0, length=90, width=20)
res = cal.measure(img)
for e in res.edges:
    print(e.t, e.point, e.polarity, e.contrast)
pair = cal.measure_pair(img, policy="first_last", pair_polarity="opposite")
```

```
print(pair.width)
```

`t`는 ROI 중심에서 측정축 방향으로 켄 위치, `point`는 같은 것의 영상 좌표다. 코어는 이게 전부다.

4. 좌표 규약 (한 번은 읽어야 한다)

- 픽셀 중심은 $(i + 0.5, j + 0.5)$. 픽셀 (0,0)의 모서리가 원점이다. 다른 라이브러리가 중심을 정수에 두고 있다면 반 픽셀이 계통적으로 어긋난다.
- 각도는 **degree**, 화면 기준 반시계가 양수 (y가 아래로 증가하므로 수학적으로는 시계 방향이지만, 보이는 대로다).
- `angle_deg`는 캘리퍼가 재는 방향이다. ROI는 그 방향으로 `length/2`씩, 수직으로 `width/2`씩 뻗는다.
- `polarity`는 측정 방향 기준으로 읽는다. `dark_to_light`는 `t`가 커질수록 프로파일이 올라간다는 뜻이다. 캘리퍼를 180° 돌리면 같은 물리적 에지의 극성이 뒤집힌다. 누구나 한 번은 여기 걸린다.
- `LineFinder`는 각 캘리퍼가 선분 `p0` → `p1`에 수직으로 켄다. 선분 방향을 +90° 돌린 방향이고, 극성은 그 방향 기준이다.
- `AnnularCaliper`와 `CircleFinder`에서 `t`는 반경이고 극성은 바깥 방향 기준이다. 어두운 배경 위의 밝은 원판이면 `light_to_dark`다.

geofit과 같은 규약이라, pose가 그대로 오간다.

5. 캘리퍼 하나, 제대로

```
cal = gauge.Caliper(
    center=(320, 240),
    angle_deg=0,          # ■■ ■■
    length=90,            # ■■■ ■■■■ ■■■ ■■
    width=20,             # ■■ ■, ■■■■ ■■
    sigma=1.5,            # ■■ ■ ■■
    contrast=0.05,        # ■■ ■■ ■■, ■■ ■■
    polarity="any",       # "dark_to_light" | "light_to_dark"
    interp="bspline3",   # ■■■■
    subpixel="gauss3",    # ■■ ■■
    project="mean",       # "mean" | "median" | "trimmed"
)
```

값 고르는 법:

파라미터	고르는 기준
<code>length</code>	부품이 어떤 위치에 있어도 에지를 담을 만큼만. 길수록 옆 에지를 잘 확률이 높다.
<code>width</code>	에지가 직선을 유지하는 만큼. 넓히면 노이즈가 $1/\sqrt{w}$ 로 줄지만, 곡면이면 $w^2/12$ 의 곡률 바이어스가 생기고, ROI가 어긋나 있으면 $w \cdot \tan\Delta$ 만큼 뭉갸다. $w \cdot \tan\Delta < 3\sigma$ 를 지킬 것.
<code>sigma</code>	1.5가 좋은 기본값. 노이즈가 많거나 에지가 무르면 올린다. 시간은 거의 안 든다. 인접 에지는 5σ 까지 간섭하므로 σ 가 곧 "두 에지가 얼마나 가까워도 되는지"를 정한다.
<code>contrast</code>	좋은 영상에서 실제 계단 높이를 재고 그 절반쯤으로.
<code>polarity</code>	줄 수 있으면 항상 줄 것. 캘리퍼가 엉뚱한 에지를 무는 걸 막는 가장 싼 방법이다.

`gauge.suggest(img)`가 영상을 읽고 `sigma`, `contrast`, `length`, `width`, `polarity`와 각각의 근거인 `why` 문자열을 제안한다. 출발점이지 정답은 아니다.

6. 결과 읽기

```
res = cal.measure(img)
res.edges          # MeasuredEdge ■■, ■■■■ ■■
res.profile         # 1D ■■ ■■■■
res.derivative      # ■■ 1■ ■■
res.t               # ■■■ ■■■■ ■■
res.flags           # ■■ ■■
res.diag            # ■■■■
```

`MeasuredEdge`에는 `t`, `point`, `index`, `amp`, `contrast`, `polarity`, `sigma_total`(총 에지 폭, 프로파일 표본 단위)이 들어 있다.

플래그는 반드시 처리해야 하는 부분이다.

플래그	뜻	할 일
<code>LOW_CONTRAST</code>	대비 문턱을 넘은 게 없다	조명을 보거나, ROI가 엉뚱한 자리에 있다
<code>NEAR_BOUNDARY</code>	에지가 프로파일 끝의 필터 마진 안에 있다	ROI를 길게 하거나 옮긴다
<code>EDGES_TOO_CLOSE</code>	두 에지가 5σ 보다 가깝다 — 피크가 서로 당긴다	σ 를 낮추거나 바이어스를 감수한다
<code>SATURATED</code>	프로파일이 선언한 포화 준위에 닿았다	노출을 줄인다. 잘린 에지는 노이즈가 아니라 편이다
<code>TILTED</code>	에지가 측정축과 수직이 아니고, 투영이 흡수할 수 있는 정도를 넘었다	ROI를 돌리거나 좁힌다

`diag`에는 `n_peaks`, `best_rejected`(대비 조건에서 탈락한 것 중 가장 강한 것 — 왜 못 찾았는지를 말해 주는 값), `sigma_total`, `psf_sigma_est`(에지 폭에서 역산한 렌즈 흐림), 그리고 `tilt_bands > 0`일 때 `tilt_deg`, `smear`, `effective_sigma`가 들어간다.

에지를 못 찾은 것은 예외가 아니다. `edges`가 빈 목록으로 돌아오고 플래그와 `best_rejected`가 채워진다. 거부가 오류가 되는 지점은 페어링이다.

```
from gaugefit import PairingError
try:
    pair = res.pair("first_last", pair_polarity="opposite")
except PairingError as e:
    log.warning("■■ ■■: %s (■■ ■■ %.3f)", e, res.diag["best_rejected"])
```

7. 어느 에지를 쓸지, 의도적으로

현장 오측정이 나오는 자리라서, 이 선택은 휴리스틱이 아니라 파라미터다.

단일 에지 — `gauge.select_edge(edges, policy)`:

정책	고르는 것
<code>first</code>	<code>t</code> 가 가장 작은 것
<code>last</code>	<code>t</code> 가 가장 큰 것
<code>strongest</code>	<code>abs(amp)</code> 가 가장 큰 것
<code>closest_to_center</code>	<code>abs(t)</code> 가 가장 작은 것

페어 — `res.pair(policy, ...)`:

정책	고르는 것
<code>first_last</code>	가장 바깥쪽 쌍
<code>strongest_pair</code>	진폭 상위 둘
<code>closest_to_nominal</code>	폭이 <code>nominal_width</code> 에 가장 가까운 쌍
<code>widest / narrowest</code>	폭의 극단

`pair_polarity="opposite"`(고체: 상승 에지 다음 하강 에지), `"same"`, 또는 `None`으로 아무거나.
`tolerance`는 `closest_to_nominal`에서 허용 범위를 벗어난 쌍을 거부한다 — 자신만만한 틀린 폭보다 오류가 낫다.

8. 직선과 원

```
line = gauge.LineFinder(
    p0=(100, 80), p1=(100, 400), # ■■■■ ■■■■ ■■
    n=15, # ■ ■
    length=40, width=10, # ■ ■■■■ ROI
    edge_policy="strongest",
    robust="tukey", ransac=True, # ■■■■ ■■
).run(img)
line.ok # ■■ ■■■ False
line.shape # Line
line.points # ■■■ ■ ■■■
line.results # ■■■■ ■■ CaliperResult, ■■ ■■, ■■■ ■■
line.used, line.failed
line.rms, line.span # span■ ■■■: inlier ■■■ peak-to-peak
line.inliers
line.flags # ■■■ ■■■ ■■■ + MANY_CALIPER_FAILURES / MANY_OUTLIERS
```

`CircleFinder`도 같은 모양이고, annular 캘리퍼를 쓴다.

```
circ = gauge.CircleFinder(center=(300, 300), r_nominal=61, n=24,
                           r_span=30, arc=8).run(img)
circ.shape.center, circ.shape.radius
circ.span # ■■■
circ.diag["curvature_correction"]
```

왜 원에 직선 ROI가 아니라 annular인가. 곡면 에지를 직선 ROI로 재면 $-(\sigma_{\text{psf}}^2 + W^2/12) / 2R$ 만큼 짧게 읽는다. 현(chord) 효과에 2D 흐림이 더해진 것이다. $R=60$, $W=20$ 이면 반경에 0.28 px, 지름으로는 0.57 px다. annular ROI는 호를 따라 투영하므로 $W^2/12$ 항이 사라진다. 남은 광학 항은 자동으로 보정한다 — 측정된 에지 폭에서 σ_{psf} 를 역산한다. 값을 알면 `psf_sigma`로 주고, 고려면 `curvature_correction=False`.

`results`는 장식이 아니다. 직선 값이 이상할 때 가장 먼저 볼 것은 어느 캘리퍼가 무엇을 냈는가다.

```
for k, r in enumerate(line.results):
    mark = "OUT" if k in line.failed else ("in" if k in line.used else " ")
    print(k, mark, r.flags, r.diag.get("best_rejected"))
```

9. 픽스처링 — 부품을 따라가기

기준 위치에 부품을 놓고 한 번 티칭한다. 런타임에는 부품이 지금 어디 있지만 알려 주면 모든 ROI가 같이 움직인다.

```
recipe = gauge.load_recipe("part.gaugefit.json")
# geofit:
out = recipe.run(img, pose=(match.x, match.y, match.angle))
# :
fx = gauge.Fixture.from_geofit(match, ref=(380.0, 260.0, 0.0))
cal_now = fx.apply(cal)
```

`pose`는 (`x`, `y`, `angle_deg`) 또는 `.x` / `.y` / `.angle`을 가진 객체다 — `geofit`의 `Match`가 그대로 들어간다. 기준 `pose`는 레시피에 저장되므로 같은 파일이 양쪽에서 동작한다.

픽스처링이 없으면 부품이 2 픽셀 밀릴 때 모든 측정값이 2 픽셀 밀린다. 있으면 상쇄된다 — 번들 샘플을 (5, 7) px 굴려도 측정값은 0.05 px 미만으로 움직인다.

10. 밀리미터

```
cal = gauge.Calibration.from_scale(px_per_unit=41.7, unit="mm")
# :
from gaugefit.calib import calibrate_from_dot_grid
cal = calibrate_from_dot_grid(target_img, rows=9, cols=11, pitch=2.0)
world = cal.to_world(points_px)
cal.distance(p_px, q_px) # :
cal.scale_at(p_px) # :/.
```

`undistort_image()`는 없고, 앞으로도 없다. 영상 왜곡보정은 리샘플링이고, 리샘플링은 캘리퍼가 읽으려는 바로 그 서브픽셀 정보를 훼손한다. `gaugefit.calib.distortion.undistort_image`는 그 문장을 담은 `NotImplementedError`를 던지려고만 존재한다. 재고 나서, 점을 보정할 것.

레시피에 캘리브레이션을 붙이면 모든 측정값이 월드 단위로 돌아오고, `out.units`가 어느 단위인지 말해 준다.

11. 설정을 레시피로 넘기기

```
gaugefit serve
```

실제 영상 위에 도구를 놓고, ROI를 끌면서 그 아래의 프로파일과 미분을 보고, 측정을 추가하고, 내보낸다. 도구·측정·픽스처·캘리브레이션·base64 참조 영상이 들어 있는 JSON 한 장이 나온다. 그 설정을 재현하는 데 더 필요한 것은 없다.

```
# 1) :
```

```

recipe = gauge.load_recipe("part.gaugefit.json")
# 2) ■■■■■■■■ ■■■■■■■■ ■■■■■■■■ ■■■■■■■■ - URL ■■■■
recipe = gauge.fetch_recipe()
# 3) ■■■■■■■■ ■■■■■■■■ JSON ■■■■■■■■
recipe = gauge.load_recipe(text)
out = recipe.run(img, pose=pose)
out.values      # {"plate_gap": 184.0, "big_dia": 80.0, ...}
out.units       # {"plate_gap": "mm", ...}
out.shapes      # ■■■■■■■■ ■■■■■■■■ Line / Circle
out.tools       # ■■■■■■■■ ■■■■■■■■
out.flags       #
out.failed      # [(■■■■, "TypeError: ..."), ...] - ■■■■■■■■ ■■■■■■■■ ■■■■■■■■

```

도구 종류: `caliper`, `annular`, `line`, `circle`. 측정식: `point_point`, `point_line`, `point_circle`, `line_line`, `line_angle`, `parallelism`, `perpendicularity`, `line_circle`, `circle_center_distance`, `circle_diameter`, `circle_radius`, `circle_roundness`, `line_straightness`, `tool_width`, `edge_position`.

`recipe.run()`이 런타임 API의 전부다. 대시보드는 설정용이고, 런타임에는 필요 없으며 라인 근처에 설치할 이유도 없다.

12. 불확실도, 그리고 그것에 관한 논쟁에서 이기기

반복도와 정확도는 다른 주장이다. 고객은 하나를 요구하면서 다른 하나를 뜻한다. 오차예산은 그 둘을 갈라 놓는다.

```

from gaugefit.quality import caliper_budget
b = caliper_budget(sigma=1.5, psf=0.8, angle=0, width=20, snr_db=35)
b.total          # ■■■ 1σ, px
for t in b.terms:
    print(t.name, t.sigma, t.kind, t.source)
# ■■■■■■■■ 0.00107 systematic ■■■ B/C (bspline3, psf 0.8, ■■■ 0°)
# ■■■■■■■■ 0.00003 systematic ■■■ A (gauss3, σ 1.5)
# ■■■■■■■■ 0.00843 random ■■■ D (gauss3, SNR 35 dB)

```

각 항이 어느 스터디에서 왔는지 이름을 달고 나온다. 그 스터디들은 [docs/bias.ko.md](https://docs.bias.ko.md)에 있고 원본 스왑은 [gaugefit/data/bias_table.csv](#)로 같이 실린다. 그래서 시비가 붙은 숫자를 주장이 아니라 측정까지 추적할 수 있다.

실제 부품으로 게이지 스터디를 하려면:

```

from gaugefit.quality import gage_rr
g = gage_rr(data, tolerance=0.05) # data[■■■][■■■■] = [■■■, ■■■, ...]
g.grr, g.ndc, g.ev, g.av, g.pv

```

13. 속도와 스레드

캘리퍼 하나(프로파일 80 × 폭 48)가 약 58 s, 강건 피팅을 포함한 24개짜리 원 파인더가 약 0.16 ms다. 시간은 리샘플링에 몰려 있고 나머지는 그다음이다. σ 를 1.5에서 6으로 올려도 6 s다. 거기서 아까지 말 것.

- ctypes 호출이 GIL을 놓는다. 여러 파이썬 스레드에서 여러 영상에 파인더를 돌리면 실제로 병렬로 돈다.

- `gauge.set_num_threads(n)`이 캘리퍼 풀을 정한다. 스레드 수는 속도만 바꾼다. 1, 2, 4, 8, 32에서 결과가 동일하고, 그걸 확인하는 시험이 있다.
- 한 프레임을 여러 도구가 볼 때는 준비한 영상을 재사용한다.

```
pre = gauge.prepare(img, "bspline3")
a = cal_a.measure(img, pre)
b = cal_b.measure(img, pre)
```

`recipe.run()`은 알아서 그렇게 한다.

- `interp="bilinear"`가 더 빠르고 위상 의존 바이어스로 약 0.03 px를 치른다. `bicubic`은 `bilinear` 대비 얻는 게 없다 — 가정이 아니라 실측이다.

14. 뭔가 이상할 때

"엉뚱한 에지를 잡았다." `polarity`를 준다. 그다음 `length`를 좁힌다. 그다음 기본값 대신 `edge_policy /` 페어링 정책을 명시한다.

"꼭이 일정하게 몇 백분의 1 어긋난다." 곡률(annular을 쓸 것), ROI 어긋남 (`diag["tilt_deg"]`), 또는 측정렬된 에지 — 0°가 리샘플링 바이어스의 최악 조건이지 최선이 아니다. ROI를 5° 기울이면 한 자릿수 줄어든다.

"노이즈가 심하다." σ 를 올리기 전에 `width`를 넓힐 것. 에지가 직선인 한 바이어스 없이 노이즈만 $1/\sqrt{w}$ 로 준다.

"두 에지가 너무 가깝다." 3σ 가 아니라 5σ 까지 간섭한다. σ 를 낮추거나, 캘리퍼를 따로 쓸 것.

"아무것도 못 찾는다." `res.diag["best_rejected"]`가 대비 조건에서 탈락한 가장 강한 후보다. `contrast`에 가까우면 문턱값 문제고, 0에 가까우면 ROI가 에지 위에 없다.

"네이티브 라이브러리를 못 찾는다." `gauge.engine_info()["load_error"]`가 이유를 말한다. 설치된 훅이 아니라 소스 체크아웃에서 돌리고 있을 가능성이 높다. 순수 파이썬 엔진은 그대로 돈다 — `gauge.use_native(False)` — 느릴 뿐이다.

"두 엔진이 같은 값을 내나?" 내야 한다. `cal.measure(img)`와 `cal.measure_py(img)`가 동일해야 하고, 아니라면 영상과 함께 신고할 만한 버그다.

15. API 요약

```
import gaugefit as gauge
# ■■
gauge.engine_info(); gauge.use_native(False); gauge.set_num_threads(8); gauge.native_available()
# ■■
gauge.Caliper(center, angle_deg, length, width, **params).measure(img) -> CaliperResult
gauge.AnnularCaliper(center, r_min, r_max, angle_deg, arc, **params).measure(img)
gauge.LineFinder(p0, p1, n, length, width, **params).run(img) -> FinderResult
gauge.CircleFinder(center, r_nominal, n, r_span, arc, **params).run(img)
gauge.prepare(img, "bspline3") # ■ ■■■■■ ■■ ■■■ ■■
# ■■ ■■
gauge.select_edge(edges, "strongest"); res.pair("first_last", pair_polarity="opposite")
gauge.SINGLE_POLICIES, gauge.PAIR_POLICIES, gauge.PairingError
# ■■ ■■ ■■
gauge.fit_line_robust(pts); gauge.fit_circle_robust(pts)
gauge.fit_line(pts); gauge.fit_circle(pts) # ■■■■■ ■■ ■■ ■■ ■■
```

```

# ■■■■■
from gaugefit import metrology as mt
mt.point_point, mt.point_line, mt.line_line, mt.circle_circle,
mt.parallelism, mt.perpendicularity, mt.intersect, mt.angle_between
# ■■■■■ ■■■■■
gauge.Fixture.from_geofit(match, ref); gauge.Fixture.from_poses(ref, cur)
gauge.Calibration.from_scale(px_per_unit, unit="mm")
from gaugefit.calib import calibrate_from_dot_grid
# ■■■ - ■■ ■■■ ■■ ■■■■■ ■■■
gauge.load_recipe(path_or_text); gauge.fetch_recipe(); recipe.run(img, pose=None)
gauge.dashboard_url()
# ■■
from gaugefit.quality import caliper_budget, gage_rr, repeatability, default_table
# ■■■ ■■■
gauge.imread(path); gauge.imwrite(path, arr); gauge.overlay(img, result); gauge.to_rgb(img)
gauge.suggest(img)
# ■■■■
gauge.license_status(); gauge.license_warnings(); gauge.TrialExpired

```